

A* (A-Star) Algorithm

Aim

To implement the **A*** (A-Star) algorithm to find the shortest path between a start node and a goal node using heuristic values.

Objectives

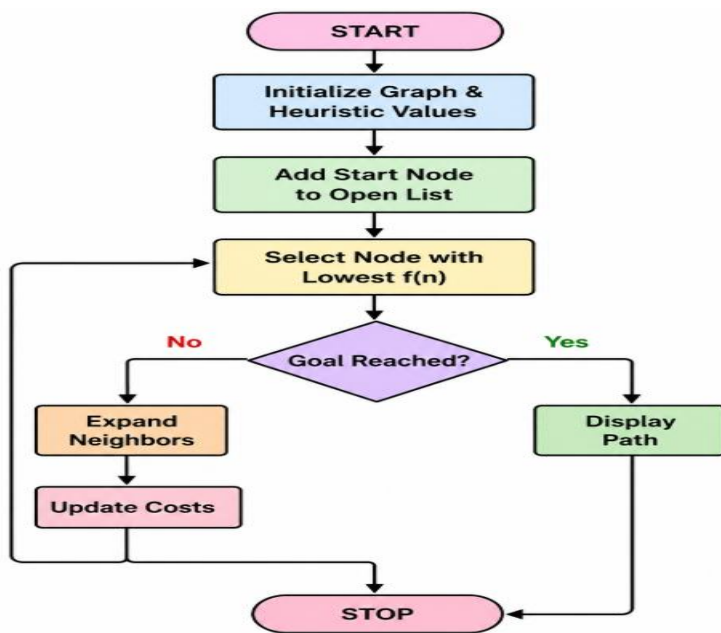
- To understand informed search algorithms.
- To find the shortest path using heuristic information.
- To minimize the total path cost.
- To implement the A* algorithm using Python.

Algorithm

1. Start.
2. Initialize the graph and heuristic values.
3. Insert the start node into the open list.
4. Select the node with the lowest $f(n) = g(n) + h(n)$.
5. If the selected node is the goal node, display the path.
6. Otherwise, expand its neighboring nodes.
7. Update the cost of each neighbor.
8. Repeat Steps 4–7 until the goal is reached.
9. Stop.

\

FLOWCHART



PROGRAM

```

import heapq

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 2)],
    'D': [],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 4,
    'E': 1,
    'F': 2,
    'G': 0
}

def astar(start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost
  
```

OUTPUT

```
> = RESTART: C:/Users/ATHIN/AppData/Local/Pr  
ssssssssssssssssssssssssssssssssssssssssss.py  
Shortest Path: A -> C -> F -> G  
> |
```

Result

The **A*** algorithm was successfully implemented and found the shortest path from the start node to the goal node using heuristic values.